One Person Lab

OPL Flow 白皮书

从个人使用 AI，到组织 AI 持续工作

OPL Flow 面向 Agent 时代构建统一的任务协同层，将人的目标、智能体能力与执行资源组织成持续工作的体系，让任务有序进入、自主推进、跨环境接力，并沉淀为可复查的成果。

Router / Worker / Fleet / Ledger

2026-11-02

Public whitepaper

目录

1 把智能能力转化为持续工作的能力 2
2 为什么需要 OPL Flow 2
3 统一管理什么，自动推进什么 2
4 四种责任组成一条工作线 3
5 Router：把注意力放到正确的工作上 3
6 Worker：自主推进，留下可接手的成果 4
7 Fleet：让连续性与容量成为共享能力 4
8 Ledger：让目标长于执行者 4
9 一个场景：从本机修改到另一台机器继续 4
10 这些分工为什么有助于提高效率 5
11 让人的注意力回到目标与判断 5
12 从小处开始，在使用中完善 5
13 在 One Person Lab 中的位置 5
14 延伸阅读 6

发布日期：2026-11-02

最近修订：2026-11-02

适用对象：希望理解 OPL Flow 工作方式的用户、合作者与开发者。

核心判断：OPL Flow 面向 Agent 时代构建统一的任务协同层，将人的目标、智能体能力与执行资源组织成持续工作的体系，让任务有序进入、自主推进、跨环境接力，并沉淀为可复查的成果。

1 把智能能力转化为持续工作的能力

模型能力正在快速增长。AI 可以理解更复杂的问题，使用更多工具，也能独立完成更长的工作。个人与小团队因此有机会组织远超自身时间与精力的执行能力，同时推进开发、研究、创作与日常运营。

这种能力要成为稳定的生产力，还需要一种持续组织工作的方式。目标需要被理解，任务需要找到负责人，执行需要获得上下文与资源，结果需要回到原来的工作线。当这些环节依赖人逐一衔接时，人的注意力就成为整套系统的瓶颈。执行能力越丰富，协调成本越值得系统性解决。

OPL Flow 的愿景，是让一个人能够组织一支持续工作的智能体团队。用户提出目标、表达偏好并作出关键判断，系统在授权范围内承担接收、投递、跟进、恢复与收口，让分散的智能能力形成连贯的工作能力。

OPL Flow 将这一愿景落实为统一的任务协同层：以一致的管理方式承接不同来源的工作，以事件推动任务流转，以可恢复的执行连接时间与机器，以唯一的任务记录维持责任和进展。它追求的结果，是人的目标能够持续转化为成果，同时减少对人的协调与注意力消耗。

2 为什么需要 OPL Flow

AI 已经可以阅读材料、编写代码、调用工具和修改文件。随着可用能力增加，一个人可以同时推进更多事情：修复软件、整理资料、检查运行状态、撰写文档，或跟进一个持续数周的项目。工作可能来自一次对话、一条审阅意见、一个外部事件，也可能来自已经授权的定时任务。

这时，用户承担的协调工作也在增加。每个任务要决定交给哪个 Agent、提供哪些背景、在哪台机器上运行；执行后还要追问进度、寻找文件、判断是否完成；遇到中断，再重新说明上下文。即使执行者越来越能干，任务之间仍可能存在很大差异：有的能自动推进，有的需要反复提醒，有的换个会话就失去连续性。

OPL Flow 要解决的就是这层协调成本。它提高任务的均一化、自动化投递与管理水平，让不同来源、不同类型的任务沿着一致的管理方式进入工作：理解目标，找到负责人，准备所需上下文与资源，投递给执行者，接收结果，并确定下一步。用户因此可以把更多精力留给目标、取舍与成果判断。

这里的“均一化”是工作管理方式的一致：任务都有清楚的目标与归属，进展能够找到，结果能够回读，中断后能够继续。研究、开发与创作仍然使用各自合适的方法，Worker 保留完成专业工作的判断空间。

“自动化投递”则是让系统在已有授权内承担日常协调：把新工作交给合适的执行者，把补充要求交给当前负责人，把等待条件恢复后的任务接回执行。需要用户决定时，系统交回一个具体的问题；其余已授权的工作继续向前。

OPL Flow 是为 Codex 提供工作基线与持久协同能力的可选 Package。精简的用户 Profile、能力建议和可复用方法支持日常工作；需要长期推进时，Ledger 保存目标与进展；需要跨机器接力或受保护的计算资源时，Fleet 提供连续性和容量。用户可以从一台电脑上的一次任务开始，随着工作需要逐步使用这些能力。

3 统一管理什么，自动推进什么

任务的内容可以千差万别，接收、交接、跟进和恢复却有共同需要。OPL Flow 围绕这些共同环节建立一致的工作方式。

工作环节	一致的管理方式	用户得到的帮助
接收	理解目标、已有成果与授权，识别新任务或现有任务的更新	少一次重复建任务与解释背景
投递	根据当前负责人、工作方法与实际能力选择执行入口	少一次手工选择 Agent 和工具

工作环节	一致的管理方式	用户得到的帮助
执行	交清上下文、预期成果和工作范围，由 Worker 自主推进	连贯完成一段有意义的工作
跟进	记录成果、阻塞与下一步，将有意义的变化交回负责人	随时知道工作在哪里、接下来由谁做
恢复	读取检查点与实际执行结果，继续同一目标	离开、换会话或换机器后容易接上
收口	对照目标检查实际成果，保存证据与剩余事项	明白已经完成什么、还有什么待处理

这种一致性让自动化有了可靠的衔接点。一次检查失败可以成为修复工作的输入，一次审阅可以成为下一轮修改的依据，一次等待结束可以成为恢复执行的条件。每次变化都能找到自己的目标与去向。

核心原则：让 Router 更快地把事件交给正确负责人，让 Worker 更容易恢复，让 Fleet 更准确地控制容量；Ledger 始终保留一份任务真相。

4 四种责任组成一条工作线

OPL Flow 用 Router、Worker、Fleet 和 Ledger 描述四种协作责任。

角色	要回答的问题	承担的责任
Router	现在该关注什么，交给谁？	理解事件、选择工作方法、找到当前负责人、决定下一步
Worker	这一段工作怎样完成？	阅读上下文、调用工具、执行任务，留下成果和继续线索
Fleet	在哪里继续，当前能承担多少工作？	准备可用环境、检查节点、管理容量与执行接力
Ledger	同一个目标已经推进到了哪里？	保存目标、依赖、当前负责人、检查点、剩余工作和结果

四种责任可以由同一个 Codex 任务中的不同动作承担，也可以在需要时分布到多个执行者和节点。简单工作由当前执行者直接完成；持续工作连接 Ledger；资源或位置变化时接入 Fleet。

协作从事件进入 Router 开始。Router 读取已有目标与负责人，把下一段工作交给 Worker；需要执行资源或接力时，由 Fleet 提供条件。Worker 将成果、检查点或阻塞写回 Ledger，需要进一步处理的变化再交给 Router。

当前的路由入口由 Skill 和 Agent 判断承载，自动化交接随着真实使用逐步补齐。

5 Router：把注意力放到正确的工作上

Router 的价值在于理解一次变化意味着什么。用户的新要求、一次检查失败、执行结果、到期事项或审阅意见，都可能改变工作的下一步。Router 结合当前目标与负责人判断：继续已有工作、调用另一种方法、安排接力，或把需要人决定的问题交回用户。

一个已经有人推进的目标，新消息应先交给这位负责人；同一问题的多次提醒，可以合并成一次有用的更新；只有状态、成果或下一步发生了有意义的变化，才值得再次占用用户的注意力。这样，Router 帮助减少重复理解、重复启动和无效打断。

OPL Flow 当前提供三个按职责划分的入口：opl-flow 负责 Flow 的基线、Ledger 与 Fleet 操作，software-development 负责软件开发、评审、架构和交付，manage-codex-tasks 负责原生任务协调、恢复与负责人迁移。用户通常直接表达目标，由执行中的 Agent 选择相应入口；熟悉流程的用户也可以直接指定 Skill。

因此，“通过 Router 调用”意味着先找到合适的方法与负责人，再开展工作。交接确定后，Worker 可以连续推进已授权的工作，在遇到新的决定、资源需求或完成节点时，再把变化交回 Router。

6 Worker：自主推进，留下可接手的成果

Worker 承担真正的执行。它可以就在当前会话中工作，也可以由另一个任务或远端执行环境承载。它围绕清楚的目标，阅读材料、比较方案、调用工具、修改成果并验证结果。

一次执行可以持续到一个自然的交接点：完成一项改动、得到一组分析结果、形成可审阅稿件，或者发现需要补充材料的问题。切片的大小由工作决定。执行者应有足够空间完成连贯工作，普通步骤由其自主安排。

为了让工作容易恢复，Worker 在交接时留下几项真正有用的信息：已经完成什么，当前成果在哪里，哪些决定已经确定，剩余工作是什么，以及接下来如何继续。代码可以通过已保存的提交继续，文档可以通过明确版本和审阅意见继续，分析可以通过输入、输出和运行记录继续。

这些线索让下一位执行者从成果出发。原会话仍可作为背景，但接力所需的关键决定和工作位置已经清楚。换模型、换会话或换机器时，同一个目标可以继续向前。

7 Fleet：让连续性与容量成为共享能力

Fleet 负责把不同机器和执行环境组织成可用资源。一台笔记本适合交互修改，另一台常开设备适合持续构建，配有 GPU 的工作站适合计算密集任务。Fleet 结合工作需求、节点能力和当前状态，为执行提供合适的位置。

连续性来自可恢复的工作。接力时先保存已有成果与检查点，目标环境从软件提供者和正式仓库准备所需能力，再通过 Ledger 的负责人转移继续同一目标。凭据由各自环境的凭据管理方提供，工作材料按其存储与访问规则读取。

容量来自当前可用状态。已登记的机器只是候选资源。准备执行时，需要确认它当前在线、环境兼容、资源足够，并且适合承担这项工作。受保护的资源通过租约记录谁在使用、有效期多久，以及何时释放。

任务等待人工意见或外部条件时，可以保存检查点，并在执行方式允许时释放计算资源。条件恢复后，再取得可用容量继续。若执行结果尚不清楚，先查明实际进展，再决定恢复、释放或重试。

Fleet 可以利用 SSH、CI Runner 和原生 Agent 连接等已有执行方式。工作方法由 Router 与 Worker 决定，Fleet 把环境准备、资源占用和接力条件处理清楚。

8 Ledger：让目标长于执行者

Ledger 保存一项工作的持久身份。一个独立目标可以经历多个会话、执行尝试和机器，仍然对应同一条任务记录。重试和接力更新这条记录；真正独立的成果才需要另立目标。

在 OPL Flow 的持久协同中，Beads/Dolt 保存目标、依赖、当前执行负责人、检查点、剩余工作和结果。每次继续之前读取当前状态，每次取得进展后写回同一目标。同一份写入范围在同一时刻有明确的执行负责人，交接时更新这份归属。

“唯一任务真相”指的是目标与执行状态的归属。代码仍由代码仓库保存，文件仍在工作空间中，发布结果仍由发布系统证明，专业质量仍由对应领域负责人判断。Ledger 连接这些成果与证据的引用，让人能够理解整个目标推进到了哪里。

人读界面可以有多种：Obsidian + Bases 保存完整任务档案，Linear 展示选定的活跃目标，工作台呈现当前进展。它们围绕同一目标组织信息，任务的执行状态始终回到 Ledger。

9 一个场景：从本机修改到另一台机器继续

下面用一个说明性场景展示四种责任如何协作。

用户希望修复一个工具，并确认它在另一种系统上正常运行。Router 识别软件开发工作，找到当前目标与负责人。Worker 在本机追踪问题、完成修改，并保存可审阅的代码与验证结果。

验证过程中，另一种系统的检查需要远端机器。Worker 留下当前提交、待验证内容和预期结果。Fleet 检查合适的节点，确认环境与容量，然后让该验证工作在目标环境中执行。主目标继续保留在同一条 Ledger 记录里。

远端检查发现一个兼容性问题。结果回到当前负责人，Router 将问题接回修复工作。Worker 根据证据修正，并再次使用需要的验证能力。各次尝试都留下结果，后续执行者知道哪些步骤已经完成。

期间用户补充了新的要求，Router 将它交给正在负责该目标的执行者。若用户需要暂停，当前成果和剩余工作保存在 Ledger，适合释放的容量交还 Fleet。恢复时，从最近的成果继续。

最终，用户得到修复后的版本、相应验证证据和清楚的交付状态。过程中可能更换执行环境，目标与成果始终连贯。

10 这些分工为什么有助于提高效率

减少重复理解。 Router 将变化送给已有负责人，Worker 按需读取相关上下文。一次事件的意义被理解后，可以直接成为下一步工作的输入。

减少重新开始。 Worker 留下可接手的成果，Ledger 保留进展与决定。更换执行者时，已有工作可以继续使用。

减少无效占用。 Fleet 根据当前能力分配资源，等待中的任务保留恢复线索，需要执行时再使用容量。

减少状态协调。目标与进展有唯一归属，各种工作界面从这里了解执行状态，再连接对应的文件和证据。

这些是分工带来的预期收益。实际改进会体现在日常体验中：用户少重复一次背景，任务少重做一段工作，机器及时归还容量，回来时更容易接上下一步。后续改进从这些具体摩擦出发。

11 让人的注意力回到目标与判断

当任务的接收、投递、跟进与恢复可以持续运转，用户就能把注意力放在更高价值的决定上：什么目标值得追求，什么方向需要调整，哪些成果值得接受，以及何时投入更多资源。

Router 把需要关注的变化交给正确的人或执行者，Worker 持续形成成果，Fleet 处理环境与容量，Ledger 保留共同的进展依据。用户表达一次目标后，系统能够沿着这条工作线继续组织执行，并在新的决定真正影响结果时清楚地呈现出来。

统一任务管理、自动化投递和可恢复执行，共同构成一种更具延展性的个人工作方式：一个人可以组织更多智能体和资源，同时保持对目标、责任与成果的理解和掌握。

12 从小处开始，在使用中完善

OPL Flow 可以先在单机上工作。精简 Profile 保持偏好与方法一致，三个 Router Skill 帮助选择合适入口，当前 Agent 完成执行。有了需要跨会话管理的目标，再使用 Ledger；有了跨机器工作或容量需求，再使用 Fleet。

现有源码已经提供 Ledger 适配、Fleet 节点检查、workspace 管理、容量租约、执行入口和负责人迁移机制。事件的统一接入、自动合并、完整的执行交接信息，以及更广泛的自动恢复，仍需要沿实际工作逐步完善。具体实现范围由工作流架构和 Fleet 架构维护。

改进首先发生在已有分工内：交接缺少信息，就补充检查点；重复提醒影响工作，就在入口合并；节点状态不够准确，就改善实时检查。并发需求增长时，再完善排队和容量分配。每一步都服务于一个已经能够说明的工作问题。

可复用的改进进入 Flow 的 Profile、Skill 或公共引擎，个人的项目、设备与策略留在私有 Instance。这样，一次实际使用的经验可以成为共同能力，同时保留每个人的工作方式。

13 在 One Person Lab 中的位置

OPL Framework 提供通用运行能力，并负责 Package 安装、Profile 物化与载体状态；App 提供工作界面；Cloud 扩展在线资源与协作；专业 Package 提供领域方法与质量判断。OPL Flow 作为可选的工作流 Package，为使用它的任务提供基线、方法和持久协同。

在这套关系中，Router 负责把请求交给合适的方法，Worker 运用这些能力产生成果，Fleet 在需要时连接执行资源，Ledger 记录目标进展。领域负责人继续判断成果是否满足专业要求。用户可以在熟悉的执行环境中采用 Flow，再逐步扩展工作范围。

本文记录这套设计的理念。安装、发布与节点运行情况由各自的实时回读确认；设计随使用持续完善，四种责任为每次改进提供稳定的方向。

14 延伸阅读

- One Person Lab 白皮书：复杂知识工作的整体产品思想。
- OPL Framework 白皮书：通用能力的组合与演进。
- OPL App 白皮书：连续工作在界面中的表达。
- OPL Cloud 白皮书：在线资源与协作。
- MAS 白皮书：专业领域如何围绕同一研究目标持续推进。
- OPL Flow 文档索引：当前架构、安装与操作入口。